Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерного проектирования
Кафедра проектирования информационно-компьютерных систем
Дисциплина «Объектно-ориентированное проектирование и программирование»

«К ЗАЩИТЕ ДОПУСТИТЬ»
Руководитель курсового проекта
ассистент
_____ Ларькин А.Д.
___ .____.2022

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту
на тему:

ПРОГРАММНОЕ СРЕДСТВО «ДОСКА ОБЪЯВЛЕНИЙ»

БГУИР КП 1-58 01 01 021 ПЗ

Выполнил студент группы 110902
МИШОТА Владислав
Геннадьевич

_____
(подпись студента)

Курсовой проект представлен на проверку ___ .____.2022

_____
(подпись студента)

Минск 2022

РЕФЕРАТ

БГУИР КП 1-58 01 01 021 ПЗ

Мишота В.Г. Программное средство «Доска объявлений» : пояснительная записка к курсовому проекту / В.Г. Мишота – Минск: БГУИР, 2022. – 28 с.

Пояснительная записка 28 с., 26 рис., 14 источников, 4 приложения

ПРОГРАММНОЕ СРЕДСТВО «ДОСКА ОБЪЯВЛЕНИЙ»

Цель проектирования: разработка веб-приложения, предоставляющего услуги по размещению объявлений.

Методология проведения работы: в процессе решения поставленных задач использованы принципы системного подхода, парадигма объектно-ориентированного программирования, принципы *SOLID*, паттерны проектирования программного обеспечения.

Результаты работы: был проведен анализ предметной области по теме «электронные доски объявлений», анализ рынка, разработка дизайна приложения, разработка архитектуры приложения, произведена программная реализация проекта по данной теме. Работа была выполнена с использованием языков *HTML, CSS, TypeScript, PHP*.

Программное средство предоставляет пользователям услуги по размещению и поиску объявлений, а администрации сайта – возможность модерировать контент и блокировать нарушителей.

Область применения результатов: могут быть использованы при проектировании и программировании подобных веб-приложений.

СОДЕРЖАНИЕ

Перечень условных обозначений, символов и терминов 4
Введение 4
1 Анализ исходных данных на курсовое проектирование 5
1.1 Анализ исходных данных к курсовому проекту 7
1.2 Обоснование и описание выбора языка программирования, средств разработки, используемых технологий и сторонних библиотек 8
2 Проектирование и разработка программного средства 10
2.1 Проектирование архитектуры и описание состояний программного средства 10
2.2 Проектирование и разработка графического интерфейса 15
2.3 Описание и реализация используемых в программном средстве алгоритмов 18
3 Эксплуатация программного средства 19
3.1 Ввод в эксплуатацию и обоснование минимальных технических требований к оборудованию 19
3.2 Руководство по эксплуатации программного средства 19
Заключение 24
Список использованных источников 25
Приложение А 26
Приложение Б 34
Приложение В 35

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ, СИМВОЛОВ И ТЕРМИНОВ

В настоящей пояснительной записке применяются следующие термины, обозначения и сокращения.

База данных (в дальнейшем *БД*) – это упорядоченный набор структурированной информации или данных, которые обычно хранятся в электронном виде в компьютерной системе.

Объектно-ориентированное программирование (в дальнейшем *ООП*) – методология программирования, основанная на представлении программы в виде совокупности взаимодействующих объектов, каждый из которых является экземпляром определённого класса, а классы образуют иерархию наследования.

Среда разработки – комплекс программных средств, используемый программистами для разработка программного обеспечения.

СУБД – система управления базой данных
БД – база данных
MVVM – Model-View-ViewModel
SPA – Single Page Application
API – Application Programming Interface
DOM – Document Object Model
SQL – Structured Query Language
HTML – HyperText Markup Language
CSS – Cascading Style Sheets

ВВЕДЕНИЕ

Многих пользователей интересует возможность размещения информации о желании продать или приобрести некоторый товар или услугу. Сейчас в большинстве случаев наиболее эффективным способом размещения такой информации является создание объявления в электронных досках объявлений (по сравнению, например, с расклейкой бумажных объявлений или размещении их в газетах).

Таким образом, спрос на электронные доски объявлений растет. Такие доски являются технологиями *Web* 2.0 – чем больше пользователей участвуют в работе системы, тем лучше становится взаимодействие всех пользователей с системой: появляется все больше объявлений на разные тематики, быстрее формируется рейтинг продавцов и т. п. За счет этого важным аспектом на момент реализации будет привлечение большого количества людей в сервис.

Доски объявлений клиенту наиболее привычно видеть в формате веб-приложений. Для таких приложений достаточно просто разработать модель монетизации: основным источником дохода может стать продвижение пользовательских объявлений (выделение их среди других, поднятие в поиске и т. п.) Теоретически пользователь будет достаточно охотно платить за такие услуги, так как их цель – что-либо продать, и если им удастся за небольшую доплату за рекламу продать товар/услугу дороже, то они, несмотря на затраты, могут остаться в плюсе. Также в перспективе продавать можно сопутствующие услуги вроде доставки, рассрочки и т. п.

Анализ предметной области и рынка. Необходимо удостовериться в востребованности приложения, а также оценить шансы успешного внедрения приложения при заданном бюджете. Выбор инструментов разработки. Инструменты разработки должны предоставлять все необходимые в приложении функции и позволить реализовать достаточно оптимизированное приложение, которое в дальнейшем можно будет легко поддерживать. Проектирование приложения. Одной из наиболее важных характеристик хорошего приложения является его масштабируемость. Простота масштабируемости приложения определяется на этапе проектировки. Реализация приложения. На этом этапе необходимо в соответствии с разработанной ранее архитектурой написать код. Тестирование приложения. При тестировании может возникнуть необходимость возврата к одному из прошлых этапов (и скорее всего она возникнет). На этом этапе должны быть выявлены все баги, уязвимости и т.п.

Согласно справке о проверке курсового проекта на антиплагиат, приведенной в приложении Б, курсовой проект оригинален на 78,8%.

1 АНАЛИЗ ИСХОДНЫХ ДАННЫХ НА КУРСОВОЕ ПРОЕКТИРОВАНИЕ

1.1 Анализ исходных данных к курсовому проекту

Тема проекта: «Программное средство «Доска объявлений»».

Описание системы: Данное программное средство будет представлять из себя веб-приложение (сайт). Во всяком случае пользователю такого приложения будут необходимы такие возможности, как авторизация, просмотр и поиск объявлений, просмотр чужих аккаунтов, просмотр и редактирование своего.

Описание системы: Данное программное средство будет представлять из себя веб-приложение (сайт). Во всяком случае пользователю такого приложения будут необходимы такие возможности, как авторизация, просмотр и поиск объявлений, просмотр чужих аккаунтов, просмотр и редактирование своего.

Назначение системы – предоставление пользователям услуг по размещению объявлений.

Минимально жизнеспособный продукт должен включать в себя такие возможности, как просмотр, поиск и создание объявлений, а также просмотр информации о других пользователях.

В дальнейшем в результате расширения функционала список необходимых возможностей может расшириться (например, могут быть добавлены такие возможности, как ведение переписки между пользователями, просмотр отзывов о других пользователях и т. п.), а потому следует заранее позаботиться о том, чтобы масштабировать код было просто. Для этого стоит разработать гибкую архитектуру, позволяющую добавлять новые возможности, затрагивая как можно меньше реализацию уже существующих.

Рассмотрим типичные примеры использования приложения.

Действующее лицо – человек, которому необходимо купить некоторый товар.

Вероятные проблемы:

– Пользователь не захочет регистрироваться. Решение: позволить пользователю пройти почти все (или все) этапы без регистрации и заставить зарегистрироваться только в конце.

Use Case:

– Пользователь попадает на сайт;

– Пользователь вводит свой запрос в поисковую строку;

– При желании пользователь добавляет фильтры;

– Пользователь видит список подходящих объявлений;

– Пользователь просматривает разные объявления и находит то, что удовлетворяет его запросам;

– Если автор объявления не разрешил просмотр контактных данных неавторизованным пользователям, то пользователь регистрируется;

– Пользователь вступает в контакт с создателем объявления и совершает сделку.

Теперь рассмотрим процесс взаимодействия с приложением с другой стороны.

Действующее лицо – человек, которому необходимо продать некий товар.

Вероятные проблемы:

– Пользователь не хочет выкладывать свои контактные данные в открытый доступ. Решение: добавить возможность запрета просмотра контактных данных неавторизованным пользователям; добавить возможность добавлять различные контактные данные, например, не только номер телефона, но и ссылки мессенджеры и т. п.

Use Case:

– Пользователь попадает на сайт;

– Пользователь регистрируется;

– Пользователь создает объявление;

– Пользователь ждет пока с ним свяжутся;

– Другой пользователь вступает в контакт с этим и совершает сделку.

1.2 Обоснование и описание выбора языка программирования, средств разработки, используемых технологий и сторонних библиотек

Веб-приложение – вид клиент-серверного приложения [1]. Соответственно, вся логика должна быть распределена между клиентской и серверной частью. Серверная часть в свою очередь включает в себя какую-то бизнес-логику и базу данных.

В приложении стоит реализовать толстый клиент [2]. Толстый клиент меньше зависит от сервера. Это позволит уменьшить траффик между клиентом и сервером в течение работы с приложением (хотя при этом, конечно, увеличит время загрузки), за счет чего работа с приложением станет более плавной и приятной. Также снизится нагрузка на сервер, а, следовательно, уменьшатся затраты на хостинг-услуги. Сайт будет представлять из себя *SPA*.

Для того, чтобы снизить временные затраты на реализацию клиентской части приложения, было принято решение использовать фреймворк *Vue.js*. Он значительно упрощает реализацию *SPA*, позволяет писать более декларативный код, удобно разбивать приложение на компоненты. *Vue.js* также реализовывает нечто похожее на архитектуру *MVVM* [4].

В связке с *Vue.js* будет использован *Typescript*. *TypeScript* отличается от *JavaScript* возможностью явного статического назначения типов, поддержкой использования полноценных классов (как в традиционных объектно-ориентированных языках), а также поддержкой подключения модулей, что призвано повысить скорость разработки, облегчить читаемость, рефакторинг и повторное использование кода, помочь осуществлять поиск ошибок на этапе разработки и компиляции, и, возможно, ускорить выполнение программ [5].

В качестве языка, на котором будет реализована бизнес-логика серверной части был выбран *PHP* – *C*-подобный мультипарадигмальный (поддерживающий в т. ч. ООП) язык программирования, используемый преимущественно для разработки веб-приложений [6]. По состоянию на 2021 год *PHP* занимал 79.2% рынка веб-разработки [7]. *PHP* имеет четкую специализацию по написанию бэкенда для проектов разной сложности [8]. Причины выбора именно этого языка:

– Быстрое написание кода за счет заточенности *PHP* под конкретные задачи;

– Гибкость, универсальность, кроссплатформенность;

– Большое сообщество разработчиков;

– Наличие неплохой документации [9].

В качестве СУБД будет использована *MySQL*. Причины такого выбора:

– Бесплатность [10];

– *SQL* в качестве языка запросов;

– Хорошая скорость работы [11];

– Безопасность [12].

2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

2.1 Проектирование архитектуры и описание состояний программного средства

Клиент-серверное приложение представляет из себя обменивающиеся данными клиент и сервер [13]. Это показано на рисунке 2.1.

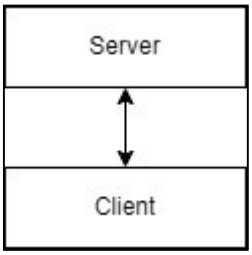


Рисунок 2.1 – Общая архитектура клиент-серверного приложения

Декомпозируем эту архитектуру. В первую очередь, сервер включает в себя *БД* и бизнес-логику, а также предоставляет *API*, к которому обращается клиент. Клиент же пока что представим в виде классического *MVVM* (рисунок 2.2).

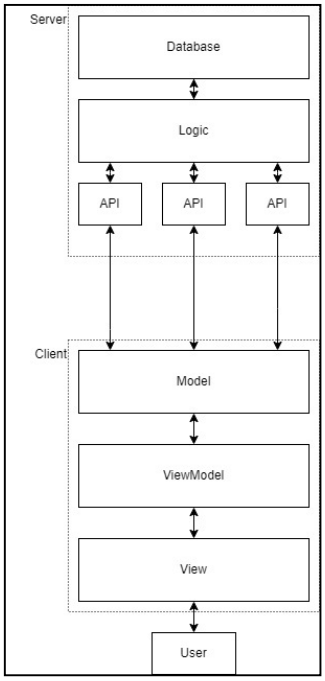


Рисунок 2.2 – Архитектура после декомпозиции сервера и клиента

В *Vue.js* используется такая концепция, как компоненты (*SFC, Single-File Component*). Они представляют собой независимые повторно используемые сочетания *HTML* кода и привязанного к нему скрипта [14].

По итогу получается, что в клиенте есть множество компонентов, в каждом из которых есть *HTML*-код – *template* (шаблон), и вместе все шаблоны по сути образуют *View* всего сайта.

В script компонента имеется возможность привязать какие-то данные к template так, чтобы при обновлении в скрипте они обновлялись в template. Получается такой локальный *MVVM*, где *template – View*, *script – Model*, а *ViewModel* – просто часть фреймворка (то есть нам не надо самим заботиться об обновлении данных, все происходит «под капотом» *Vue.js; ViewModel* на схеме означает лишь то, что мы можем связать данные в script и в template). Вид архитектуры, в которой клиент представлен множеством компонентов представлен на рисунке 2.3.

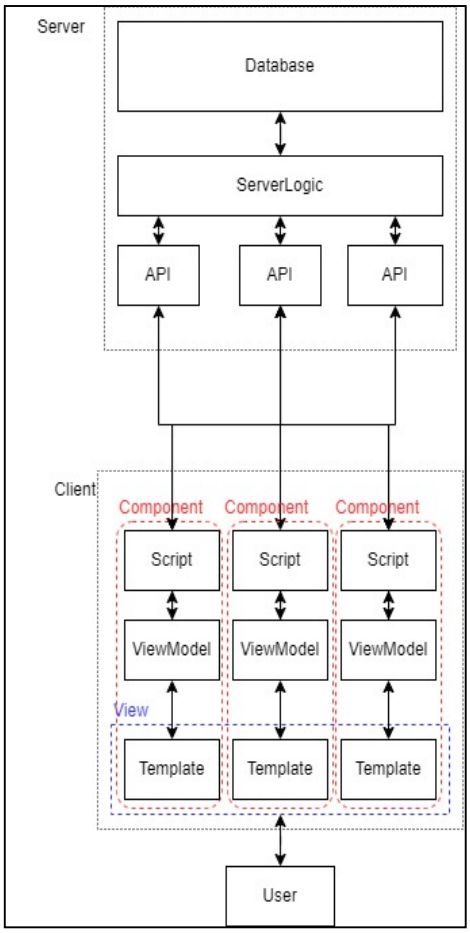


Рисунок 2.3 – Архитектура с клиентом как совокупностью компонентов

На схеме проблема заметна сразу: теперь при изменении какого-либо *API* на сервере нам придется искать все обращения к нему, которые могут быть в любом компоненте. Очевидно, что обращаться напрямую от скрипта компонента к *API* не стоит. Решением может стать создание фасада для *API* – *Model*, которая ответственна за то, чтобы получать данные от сервера (рисунок 2.4).

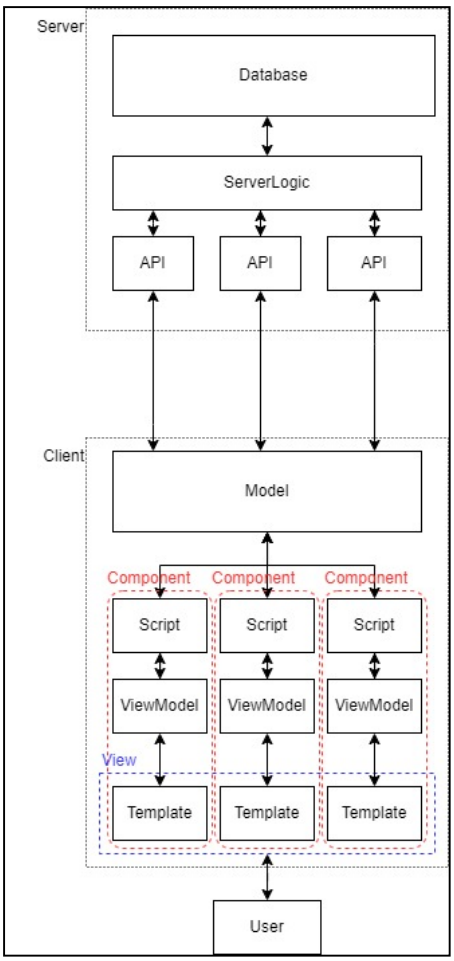


Рисунок 2.4 – Создан фасад для *API* сервера

У архитектуры все еще есть проблема. *Model* – это объект, работающий со всем, что мы получаем от сервера. *Model* очень громоздкая, и во-первых ее просто неудобно редактировать, а во-вторых меняя что-либо в ней, можно поломать что-то другое, совершенно не относящееся к объекту изменения. То же относится и к *ServerLogic*, который обрабатывает все и сразу. Эту часть программы стоило бы разбить на изолированные модули (рисунок 2.5).

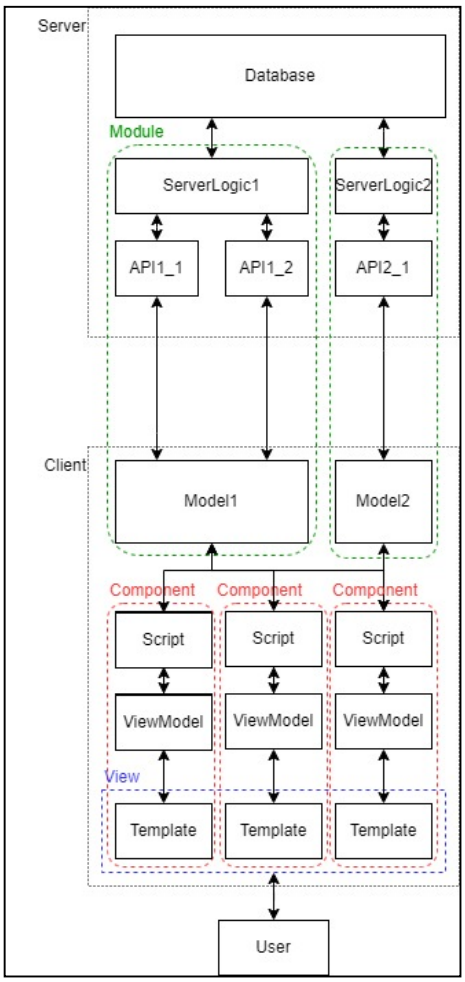


Рисунок 2.5 – *Model, API* и *ServerLogic* разбиты на изолированные модули

Конечно, иногда некоторые модули могут связываться друг с другом. Например, модуль объявлений может спросить у модуля авторизации, можно ли показывать пользователю контактные данные.

Получившаяся в итоге архитектура очень легко масштабируется.

Файловая структура – классическая для *Vue.js*: *assets*, *components*, *router*, *views. App.vue* – само приложение, которое устанавливается (mount) в *main.ts* (рисунок 2.6) .

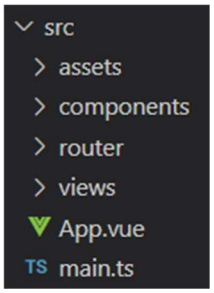


Рисунок 2.6 – Файловая структура проекта

На рисунке 2.7 приведен фрагмент диаграммы классов. Полная диаграмма классов представлена в приложении.

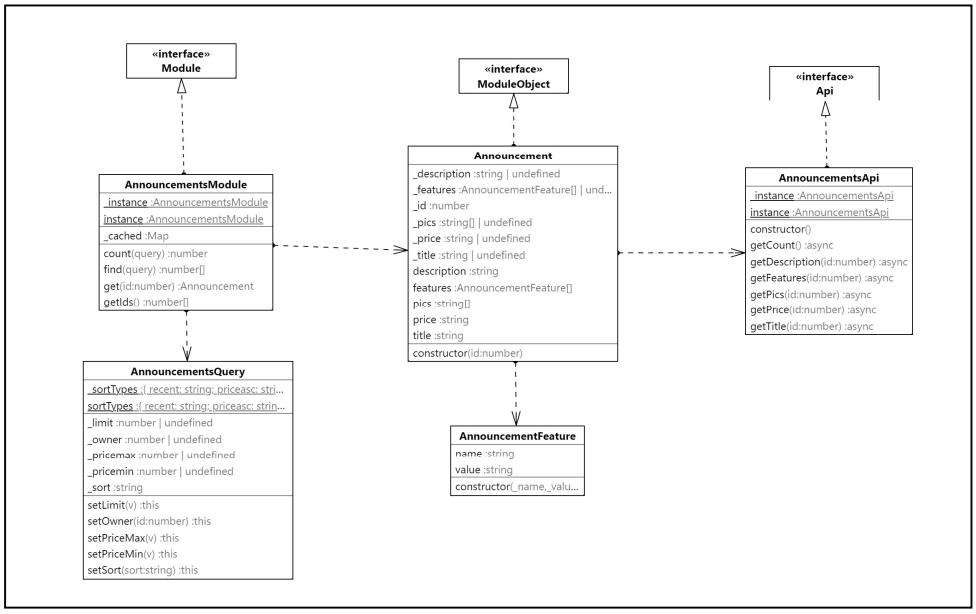


Рисунок 2.7 – Фрагмент диаграммы классов

На рисунке 2.8 приведен фрагмент диаграммы состояний. Полная диаграмма состояний представлена в приложении.

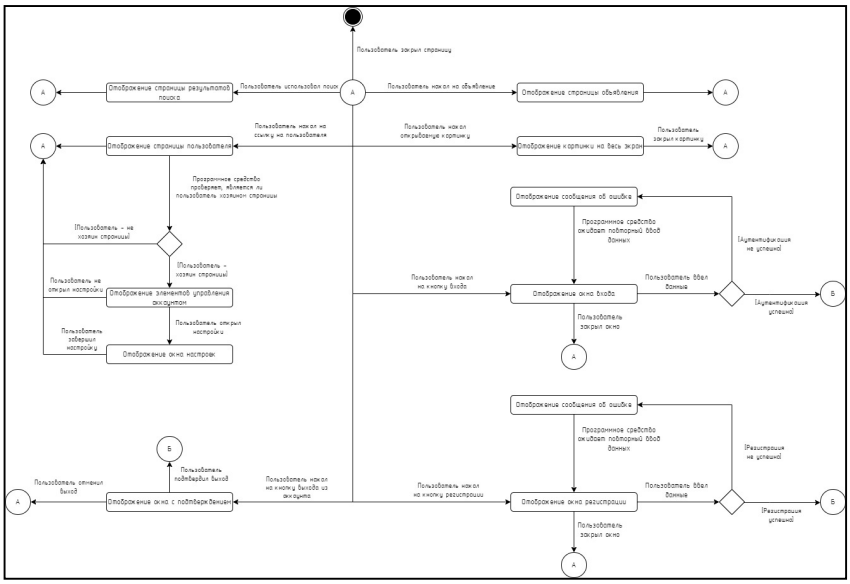


Рисунок 2.8 – Фрагмент диаграммы состояний

Таким образом, была спроектирована гибкая архитектура, позволяющая легко масштабировать код приложения.

2.2 Проектирование и разработка графического интерфейса

Пользовательский интерфейс программного средства «Доска объявлений» представлен следующими страницами:

– Главная;
– Страница результатов поиска;
– Страница пользователя;
– Страница объявления;
– Страница создания объявления.

Внешний вид страницы «Главная» представлен на рисунке 2.9.

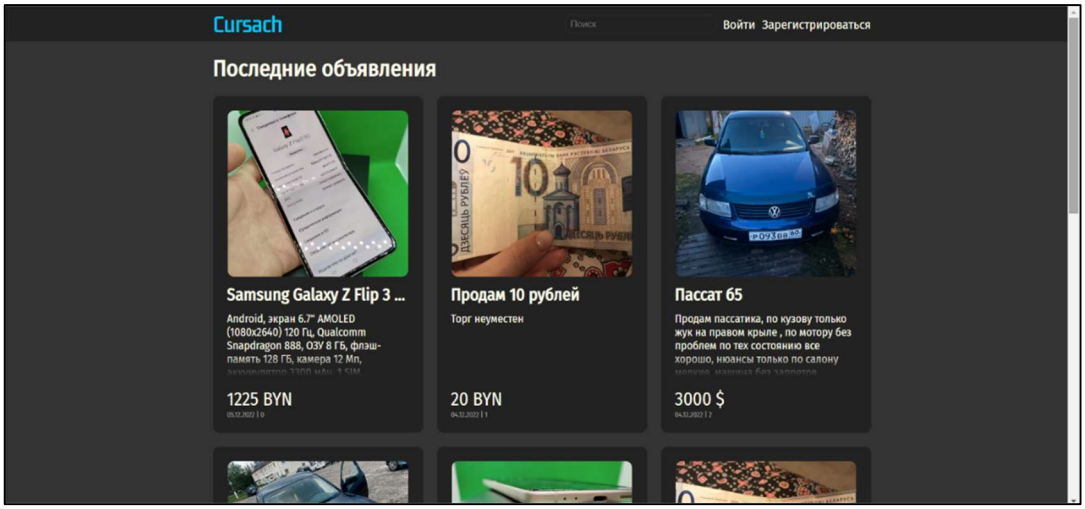


Рисунок 2.9 – Внешний вид страницы «Главная»

В хедере сайта располагаются теоретически наиболее часто используемые элементы сайта: ссылка на главную страницу, поисковик, кнопки входа/регистрации, а для авторизованных пользователей вместо последних – ссылка на свою страницу и кнопка выхода.

Объявления представлены в виде карточек, на которых есть краткая информация о них: название, изображения (листаются наведением мыши), начало описания, цена, дата создания. Более подробную информацию об объявлении можно получить, открыв его. Там он может перейти на страницу хозяина объявления, открыть изображения на полный экран (рисунок 2.10).

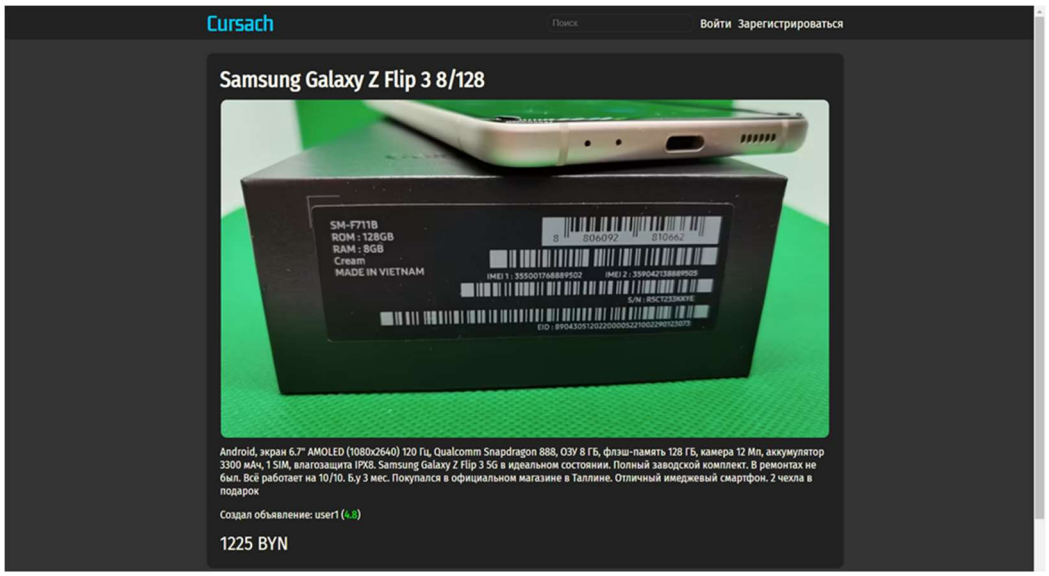


Рисунок 2.10 – Более подробная информация об объявлении

Информацию о пользователе можно получить, открыв страницу этого пользователя (рисунок 2.11).

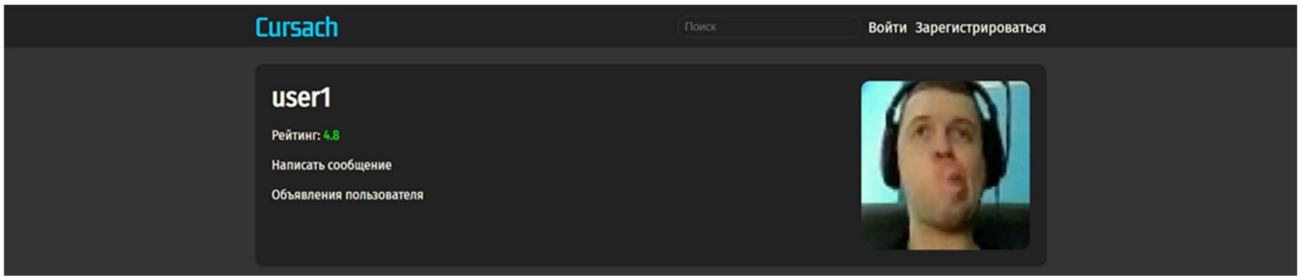


Рисунок 2.11 – Информация о пользователе

В футере есть выпадающие списки для выбора языка и темы (рисунок 2.12)



Рисунок 2.12 – Футер

Таким образом в рамках данного подраздела была произведена разработка графического интерфейса программного средства «Доска объявлений».

2.3 Описание и реализация используемых в программном средстве алгоритмов

Схема алгоритма позволяет наглядно представить способ работы приложения в графическом виде. Полная схема алгоритма работы программы представлена в приложении.

Алгоритм работы приложения начинается с загрузки главной страницы. Далее подгружается и отображается список последних объявлений. Затем программное средство проверяет, авторизован ли пользователь, и делает внешний вид меню в хедере соответствующим.

Далее все зависит от действий пользователя. Например, если пользователь нажал ссылку на чей-либо профиль, то веб-приложение отобразит соответствующий профиль, и, если пользователь авторизован как владелец профиля, позволит ему редактировать этот профиль, и снова начнет

ждать действий пользователя, который может нажать на объявление, опубликованное хозяином профиля, может захотеть авторизоваться и так далее.

В качестве примера на рисунке 2.13 показана часть схемы алгоритма.

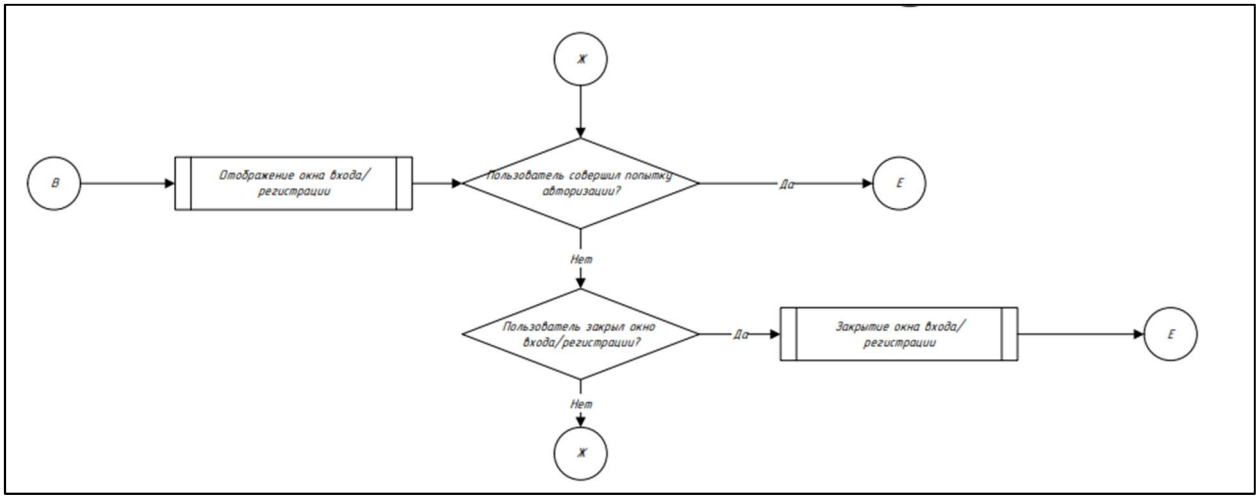


Рисунок 2.13 – Часть схемы алгоритма

Таким образом в рамках данного подраздела были произведены проектирование и разработка алгоритмов программного средства «Доска объявлений». Более точная информация находится в приложенных плакатах.

3 ЭКСПЛУАТАЦИЯ ПРОГРАММНОГО СРЕДСТВА

3.1 Ввод в эксплуатацию и обоснование минимальных технических требований к оборудованию

Веб-приложение делится на две части: клиентскую и серверную.

Требования к клиентскому оборудованию – наличие *DOM*-интерфейса (браузера) и аппаратного обеспечения для поддержания его нормальной работы [15].

Программные требования к серверной части: *PHP* 7.4, *MySQL* 8.0, *Apache*.

Аппаратные требования к серверу зависят от той нагрузки, которую должен выдерживать сервер на данном этапе эксплуатации приложения.

Таким образом, для развертывания приложения на сервере необходимо установить и настроить *Apache, PHP* и *MySQL*, а на клиенте для работы с приложением необходимо наличие современного браузера.

3.2 Руководство по эксплуатации программного средства

Когда пользователь впервые попадает на сайт, он, очевидно, не авторизован. Тем не менее, ввиду специфики сайта незарегистрированный пользователь может просматривать контент сайта (объявления, пользователи) и производить поиск.

Попав на главную страницу, как авторизованный, так и неавторизованный пользователь увидит список недавно созданных объявлений (рисунок 4.1).

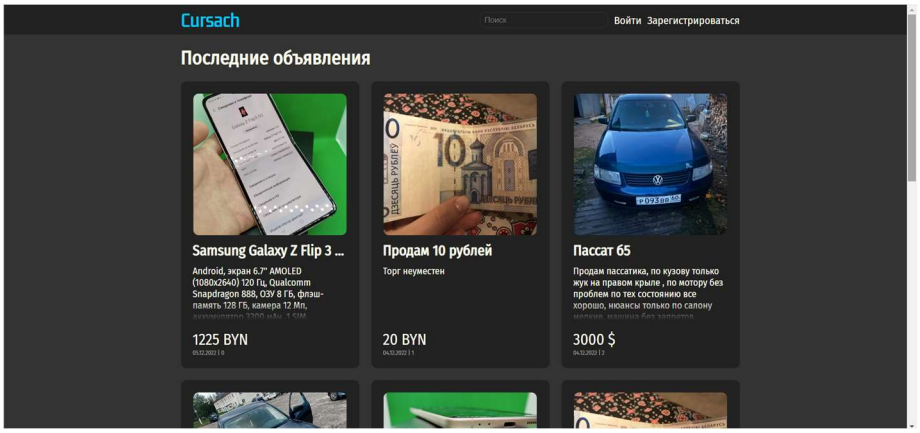


Рисунок 3.1 – Главная страница сайта

Начнем рассмотрение графического интерфейса с хедера. Слева можно заметить название проекта. При нажатии на него, пользователь попадает на главную (рисунок 1). Далее находится поисковая строка. Пользователь может ввести в нее поисковой запрос и попасть на страницу результатов поиска (рисунок 2).

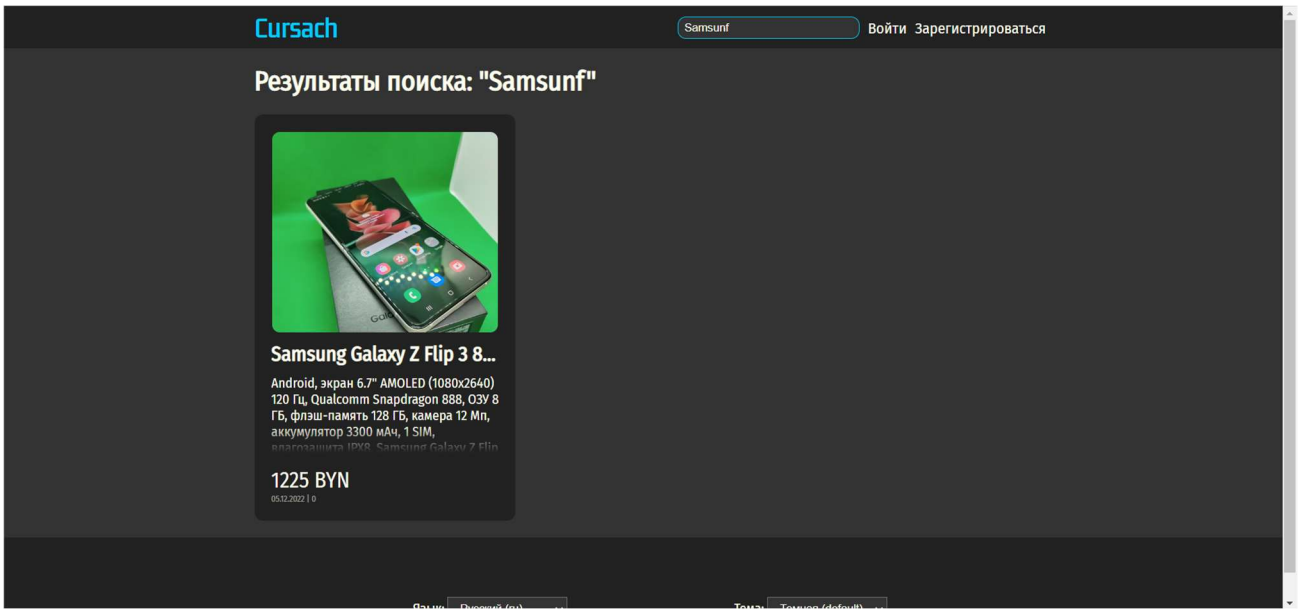


Рисунок 3.2 – Результаты поиска

Поиск ищет не только полные совпадения: так, если пользователь совершил опечатку и набрал “*Samsunf*”, будут выведены результаты в том числе содержащие “*Samsung*”. Но вернемся к хедеру. Правее поисковой строки для неавторизованного пользователя находятся кнопки «Войти» и «Зарегистрироваться», при нажатии на которые открываются соответствующие окна (рисунки 3-4).

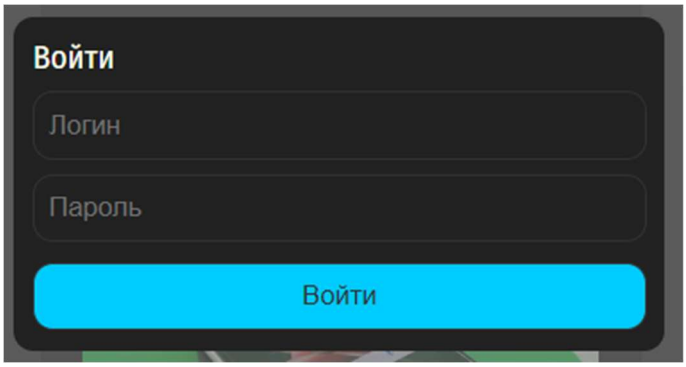


Рисунок 3.3 – Окно с формой входа

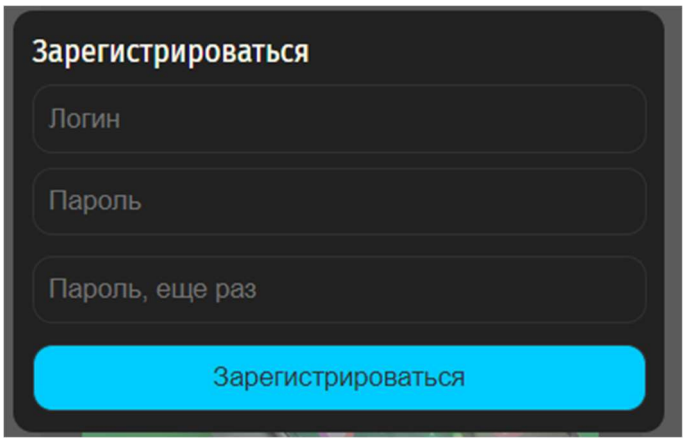


Рисунок 3.4 – Окно с формой регистрации

Для зарегистрированных пользователей в хедере указан логин, а также есть кнопки перехода к сообщениям и выхода из аккаунта (рисунок 5).



Рисунок 3.5 – Хедер для пользователя с логином “*vishota*”

Вернемся к главной странице. Там мы видим карточки объявлений. На карточке каждого объявления есть изображение. Если создавший объявление добавил несколько изображений, то при перемещении курсора по картинке они будут меняться (рисунок 6).



Рисунок 3.6 – При передвижении курсора по картинке, изображения меняются

При нажатии на одно из объявлений, мы можем просмотреть более детальную информацию о нем (рисунок 7).

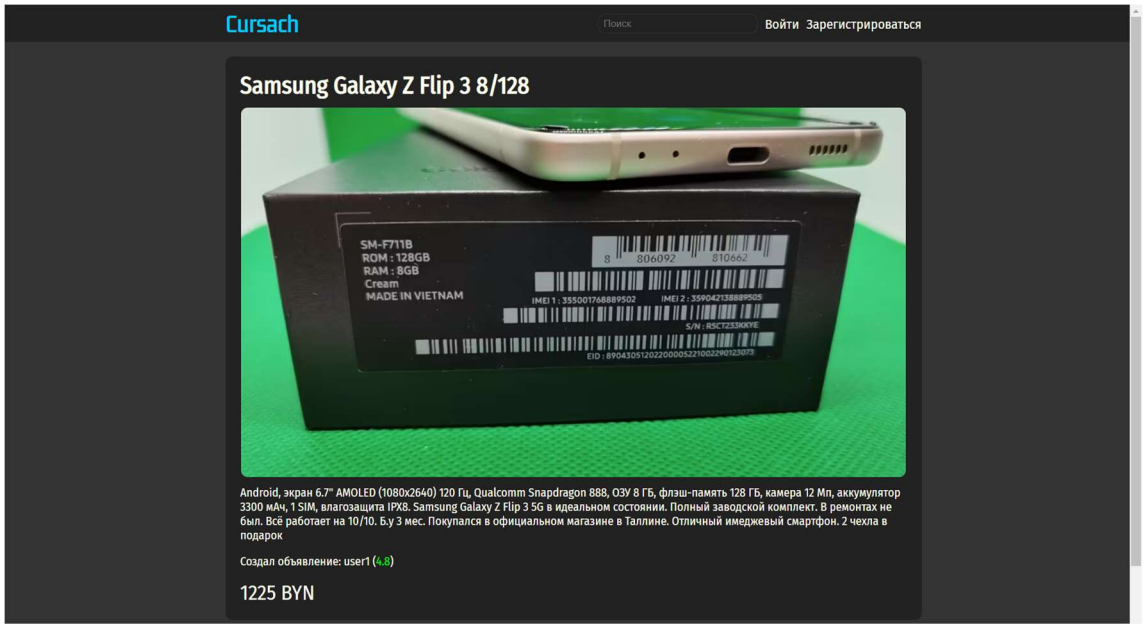


Рисунок 3.7 – Страница объявления

Здесь все изображения также можно просмотреть, двигая курсором. Нажав на логин создателя объявления, мы можем перейти в его аккаунт (рисунок 8).

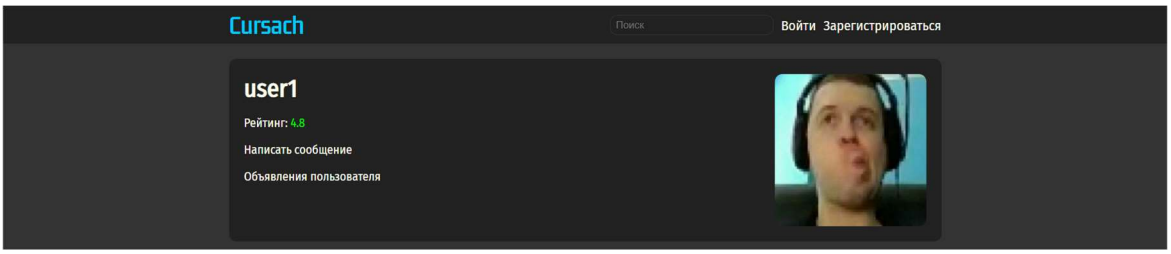


Рисунок 3.8 – Аккаунт пользователя user1

Теперь рассмотрим нижнюю часть сайта – футер (рисунок 9).



Рисунок 3.9 – Футер сайта

Здесь мы можем выбрать предпочитаемую тему. Выберем английский язык. Язык интерфейса изменился на английский (рисунок 10).

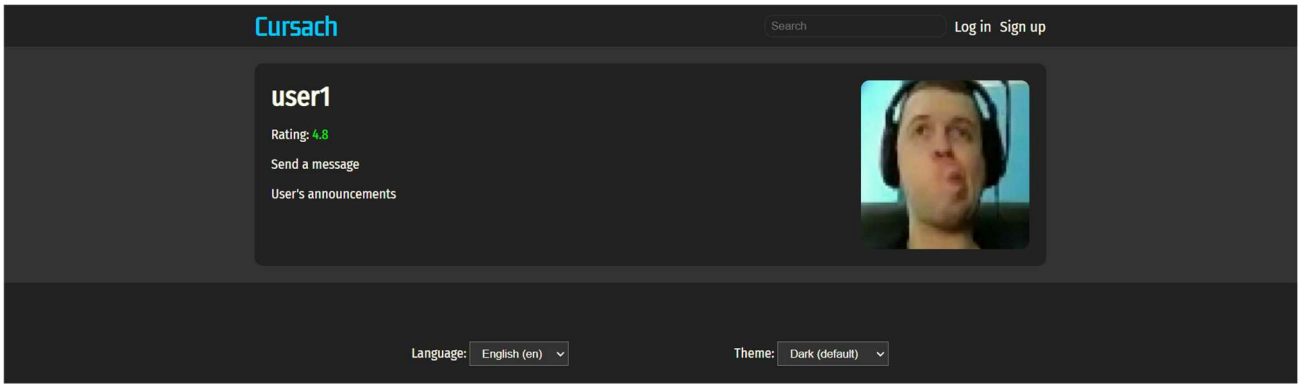


Рисунок 3.10 – Интерфейс на английском языке

Наконец, попробуем изменить тему. Выберем тему *lgreen* из выпадающего списка, и цвета приложения изменятся (рисунок 11).



Рисунок 3.11 – Тема *lgreen*

Также пользователь может открывать изображения, установленные в качестве фото профиля пользователя и изображения, прикрепленные к объявлениям путем клика левой кнопкой мыши по нужному изображению.

ЗАКЛЮЧЕНИЕ

В рамках данного курсового проекта были решены такие задачи, как анализ предметной области и рынка, выбор инструментов разработки, проектирование приложения, реализация приложения, тестирование приложения.

При решении задачи были изучены методы предъявления и анализа требований к программному обеспечению, задействованы навыки работы с *PHP, MySQL, TypeScript, HTML, CSS*, изучен фреймворк *Vue.js*. Был разработан пользовательский интерфейс, позволяющий пользователю взаимодействовать с системой наиболее эффективно.

Архитектура была разработана таким образом, что в будущем приложение можно будет достаточно просто масштабировать.

Разработка сайта как *SPA* позволила сделать работу с ним более приятной для пользователя.

Веб-приложение предоставляет обычному пользователю возможности по поиску и размещению объявлений. Администрация сайта может модерировать контент на сайте и блокировать пользователей, нарушающих правила.

Кроме прочего, в системе есть возможность выбора темы (оформления).

Ввиду востребованности и роста популярности веб-приложений, в частности веб-приложений для размещения объявлений о покупке/продаже товаров/услуг, можно сделать вывод, что проделанная работа является актуальной.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] *Testmatick* [Электронный ресурс]. – Режим доступа: https://testmatick.com/ru/ponyatie-tonkogo-i-tolstogo-klienta/. – Дата доступа: 13.10.2022.

[2] *Skillbox* [Электронный ресурс]. – Режим доступа: https://skillbox.ru/media/code/kak_sozdat_spa_na_js_i_php_za_chas/. – Дата доступа: 13.10.2022.

[3] *Medium* [Электронный ресурс]. – Режим доступа: https://medium.com/@nohkachi/my-vue-on-mvvm-c3ffb4f0678f/. – Дата доступа: 13.10.2022.

[4] *Typescript* [Электронный ресурс]. – Режим доступа: https://www.typescriptlang.org/. – Дата доступа: 13.10.2022.

[5] *Skillbox* [Электронный ресурс]. – Режим доступа: https://skillbox.ru/media/code/php_ot_istokov_do_sovremennosti/. – Дата доступа: 13.10.2022.

[5] *Kinsta* [Электронный ресурс]. – Режим доступа: https://kinsta.com/php-market-share/. – Дата доступа: 13.10.2022.

[6] *PHP* [Электронный ресурс]. – Режим доступа: https://www.php.net/. – Дата доступа: 13.10.2022.

[7] *PHP* [Электронный ресурс]. – Режим доступа: https://www.php.net/docs.php/. – Дата доступа: 13.10.2022.

[8] *MySQL* [Электронный ресурс]. – Режим доступа: https://www.mysql.com/. – Дата доступа: 13.10.2022.

[10] *Хабр* [Электронный ресурс]. – Режим доступа: https://habr.com/ru/company/simbirsoft/blog/271485/. – Дата доступа: 13.10.2022.

[11] *Oracle* [Электронный ресурс]. – Режим доступа: https://www.oracle.com/mysql/what-is-mysql/. – Дата доступа: 13.10.2022.

[12] *Хабр* [Электронный ресурс]. – Режим доступа: https://habr.com/ru/post/495698/. – Дата доступа: 13.10.2022.

[13] *Vue.js* [Электронный ресурс]. – Режим доступа: https://v3.ru.vuejs.org/ru/guide/component-basics.html. – Дата доступа: 13.10.2022.

[14] *Mozilla* [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/ru/docs/Web/API/Document_Object_Model/Introduction. – Дата доступа: 13.10.2022.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Компонент AnnouncementView

```
<script lang="ts" setup>
    import lang from '@/assets/modules/Lang/Lang';
    import announcements from '@/assets/modules/Announcements/AnnouncementsModule';
    import users from '@/assets/modules/Users/UsersModule';
    import { ref, reactive } from 'vue';
    import { useRoute } from 'vue-router';
    import GalleryComponent from '@/components/GalleryComponent.vue';
    import UserLink from '@/components/users/UserLink.vue';

    var id = ref(parseInt(useRoute().params.id as string));
    var announcement = reactive(announcements.get(id.value));

    function featureToString(value: boolean|string) {
        if(value === true) return lang.get().yes;
        if(value === false) return lang.get().no;
        return value;
    }
</script>
<template>
    <div class="vertflex">
        <div class="upper block">
            <div class="gallery-container">
                <GalleryComponent :pics="announcement.pics" class="gallery" openable />
            </div>
            <div class="vertflex">
                <h1 class="userselect">{{announcement.title}}</h1>
                <p class="userselect">{{announcement.description}}</p>
                <UserLink id="1"/>
                <h3>{{announcement.price}}</h3>
            </div>
        </div>
```

```
        <div class="block">
            <table class="features userselect">
                <tr v-for="feature in announcement.features">
                    <td>{{lang.get().features.get(feature.name)}}</td>
                    <td>{{featureToString(feature.value)}}</td>
                </tr>
            </table>
        </div>
    </div>
</template>
<style scoped>
    .gallery-container {
        width: 100%;
    }
    .upper.block {
        display: grid;
        grid-template-columns: 2fr 1fr;
        height: fit-content;
        gap: var(--s-base-indent)
    }
    .gallery {
        border-radius: var(--s-border-radius);
    }
    h1,h3 {
        margin: 0;
    }
    p {
        margin: 0;
    }
    .features {
        width: 100%;
    }
    .features tr td:first-child {
        width: 40%;
    }
</style>
```

Класс UsersModule

```
class UsersModule {
    const TOKEN_LENGTH = 30, TOKEN_SYMBOLS =
'qwertyuiopasdfghjklzxcvbnmQWERTYUIOPASDFGHJKLZXCVBNM1234567890';

    use SingletonTrait;

    function emailSignUp(string $username, string $password, string
$email) { // : int|bool (available in php 8.0)
        CursachDatabase::getInstance()->beginTransaction();
        $accountInsert = CursachDatabase::getInstance()->request('INSERT
INTO accounts (email, password_hash) VALUES (:email, :password_hash)',
            [ 'email' => $email, 'password_hash' =>
password_hash($password, null) ]);
        if($accountInsert->rowCount() === 0) {
            CursachDatabase::getInstance()->rollBack();
            return false;
        }
        $id = $accountInsert->lastInsertId();
        $profileInsert = CursachDatabase::getInstance()->request('INSERT
INTO profiles (account_id, username) VALUES (:account_id, :username)',
            [ 'account_id' => $id, 'username' => $username ]);
        if($profileInsert->rowCount() === 0) {
            CursachDatabase::getInstance()->rollBack();
            return false;
        }

        CursachDatabase::getInstance()->commit();
        return $id;
    }

    function emailSignIn(string $email, string $password) {
        $hashRequestResult = CursachDatabase::getInstance()
            ->request('SELECT id, password_hash FROM accounts WHERE email
= ?', [$email]);
        if($hashRequestResult->rowCount() == 0) return false;
        $hash = $hashRequestResult
            ->response()[0]['password_hash'];
        return (password_verify($password, $hash) ? $hashRequestResult-
>response()[0]['id'] : false);
```

```
        }

        function issueToken(int $id) {
            $code = '';
            for($i = 0; $i < self::TOKEN_LENGTH; $i++) {
                $code .= self::TOKEN_SYMBOLS[rand(0, strlen(self::TOKEN_SYMBOLS) - 1)];
            }
            setcookie('id', $id, ['expires' => time() + strtotime('+1 year'), 'path' => '/']);
            setcookie('token', $code, ['expires' => time() + strtotime('+1 year'), 'path' => '/']);
            CursachDatabase::getInstance()
                ->request('INSERT INTO tokens (account_id, token) VALUES (?,?)', [$id, $code]);
        }
        function checkToken() {
            $id = $_COOKIE['id'];
            $token = $_COOKIE['token'];
            var_dump($id);
            var_dump($token);
            return CursachDatabase::getInstance()
                ->request('SELECT * FROM tokens WHERE account_id = ? && token = ?', [$id, $token])
                ->rowCount() !== 0;
        }

        function isEmailAvailable(string $email) {
            return CursachDatabase::getInstance()
                ->request('SELECT id FROM accounts WHERE email = ?', [$email])
                ->rowCount() === 0;
        }
        function isUsernameAvailable(string $username) {
            return CursachDatabase::getInstance()
                ->request('SELECT account_id FROM profiles WHERE username = ?', [$username])
                ->rowCount() === 0;
        }
        function isUsernameAllowed(string $username) {
```

```
            return preg_match('/[a-zA-Z0-9_]{4,16}/', $username) !== 0;
        }
        function isPasswordAllowed(string $username) {
            return         preg_match('/[a-zA-Z0-9!@#$%^&*()_+=\-.]{8,32}/',
$username) !== 0;
        }

        function getUsername(int $id) {
            return CursachDatabase::getInstance()
                ->request('SELECT username FROM profiles WHERE account_id =
?', [$id], PDO::FETCH_NUM)
                ->response()[0];
        }
        function getPicture(int $id) {
            return CursachDatabase::getInstance()
                ->request('SELECT picture FROM profiles WHERE account_id =
?', [$id], PDO::FETCH_NUM)
                ->response()[0];
        }
    }
```

Класс AnnouncementsModule

```
      class AnnouncementsModule {
            use SingletonTrait;

            // method that gets value from query
            private function getFromQuery(string $query, string $param,
string $type) {
                preg_match("/$param=(?P<$param>$type+)/", $query, $matched);
                return (isset($matched[$param]) ? $matched[$param] : null);
            }
            // method that makes connects statements with AND
            private function makeWhereStatement(array $conditions) : string
{
                $statement = '';
                foreach ($conditions as $condition) {
                    $statement .= "$condition AND ";
                }
```

```
            return mb_strcut($statement, 0, -5);
        }
        // method that transforms client query to database request
        private function queryToRequest(string $query) : array {
            $conditions = [];
            $params = [];

            $text = self::getFromQuery($query, 'text', '[^\|]');
            if($text !== null) {
                array_push($conditions, 'MATCH (title, description)
AGAINST (:text)');
                $params['text'] = $text;
            }
            $pricemin = self::getFromQuery($query, 'pricemin', '\d');
            if($pricemin !== null) {
                array_push($conditions, 'price >= :pricemin');
                $params['pricemin'] = $pricemin;
            }
            $pricemax = self::getFromQuery($query, 'pricemax', '\d');
            if($pricemax !== null) {
                array_push($conditions, 'price <= :pricemax');
                $params['pricemax'] = $pricemax;
            }
            $owner = self::getFromQuery($query, 'owner', '\d');
            if($owner !== null) {
                array_push($conditions, 'owner_id = :owner');
                $params['owner'] = $owner;
            }
            $limit = self::getFromQuery($query, 'limit', '\d');
            if($limit !== null) {
                array_push($conditions, 'limit = :limit');
                $params['limit'] = $limit;
            }
            $sort = self::getFromQuery($query, 'sort', '\d');
            switch($sort) {
                case 'recent': $sort = 'ORDER BY created DESC'; break;
                case 'priceasc': $sort = 'ORDER BY price ASC'; break;
                case 'pricedesc': $sort = 'ORDER BY price DESC'; break;
```

```
                    case 'relevance':default:  $sort = 'ORDER BY relevance
DESC'; break;
                }
                $select = 'SELECT id' . ($text ? ', MATCH (title, description)
AGAINST (:text) AS relevance' : ', created AS relevance') . ' FROM
announcements';
                $whereStatement = self::makeWhereStatement($conditions);
                if($whereStatement != '') $whereStatement = 'WHERE ' .
$whereStatement;
                return [
                    'statement' => "$select $whereStatement $sort",
                    'params' => $params
                ];
            }

      function search(string $query) {
                $request = $this->queryToRequest($query);
                //var_dump($request);
                $requestResult      =      CursachDatabase::getInstance()-
>request($request['statement'], $request['params'])->response();
                $ids = [];
                foreach ($requestResult as $key => $value) {
                    array_push($ids, $value['id']);
                }
                return $ids;
            }
            function getTitle(int $id) : string {
                return CursachDatabase::getInstance()
                    ->request('SELECT title FROM announcements WHERE id=?',
[$id])
                    ->response()[0]['title'];
            }
            function getDescription(int $id) : string {
                return CursachDatabase::getInstance()
                    ->request('SELECT description FROM announcements WHERE
id=?', [$id])
                    ->response()[0]['description'];
            }
            function getOwner(int $id) : int {
```

```
                return CursachDatabase::getInstance()
                    ->request('SELECT owner_id FROM announcements WHERE
id=?', [$id])
                    ->response()[0]['owner_id'];
            }
            function getFeatures(int $id) : string {
                return CursachDatabase::getInstance()
                    ->request('SELECT features FROM announcements WHERE
id=?', [$id])
                    ->response()[0]['features'];
            }
            function getPics(int $id) : string {
                return CursachDatabase::getInstance()
                    ->request('SELECT pictures FROM announcements WHERE
id=?', [$id])
                    ->response()[0]['pictures'];
            }
            function getPrice(int $id) : string {
                return CursachDatabase::getInstance()
                    ->request('SELECT price FROM announcements WHERE id=?',
[$id])
                    ->response()[0]['price'];
            }

            function addAnnouncement(int $owner_id, string $title, string
$description, int $price, string $featuresJson, string $picturesJson) {
                return CursachDatabase::getInstance()->request(
                    'INSERT INTO announcements (id, owner_id, title,
description, price, created, features, pictures)
                    VALUES (NULL, ?, ?, ?, ?, CURRENT_TIMESTAMP, ?, ?)',
                    [$owner_id, $title, $description, $price, $featuresJson,
$picturesJson])
                        ->lastInsertId();
            }
        }
```

ПРИЛОЖЕНИЕ Б
(обязательное)
Справка о проверке курсового проекта на плагиат

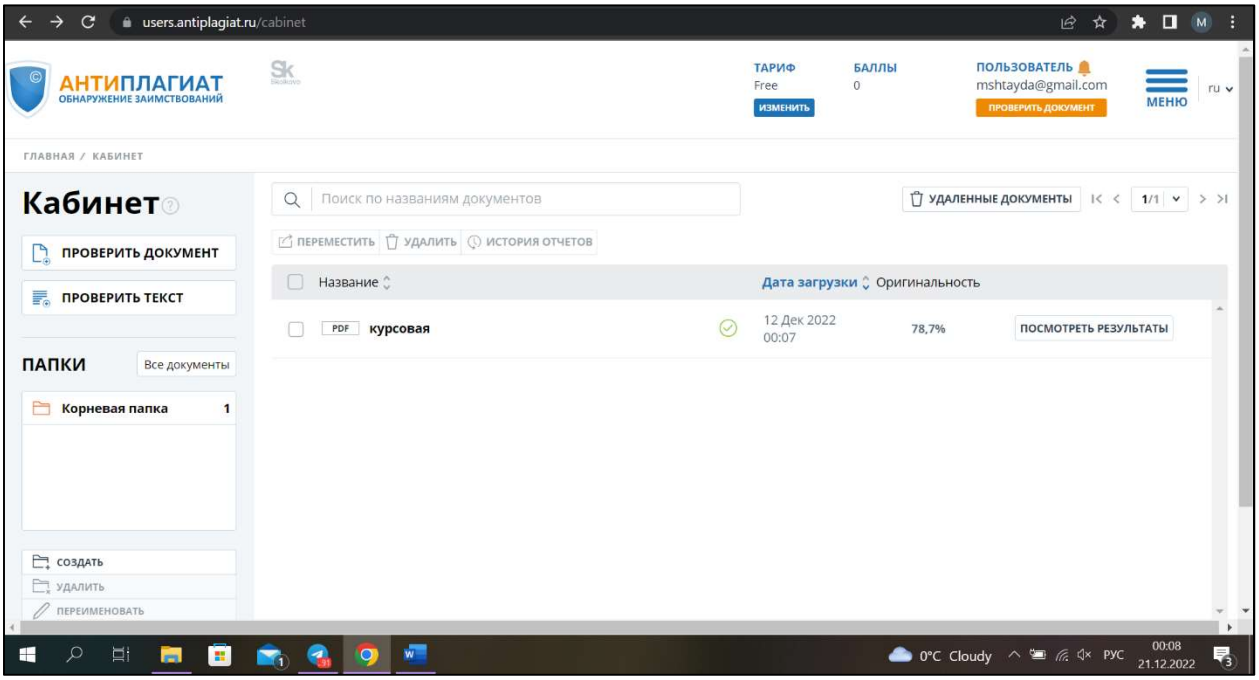


Рисунок Б.1 – Результаты проверки на оригинальность

ПРИЛОЖЕНИЕ В
(обязательное)
Ведомость